

Algoritmos Recursivos

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Definições Recursivas e Processos

- Na matemática, vários objetos são definidos apresentando-se um processo que os produz
 - Por exemplo, π é definido como a razão entre a circunferência de um círculo e seu diâmetro
- Isso equivale ao seguinte conjunto de instruções: (i) obter a circunferência de um círculo e seu diâmetro, (ii) dividir o primeiro pelo último e (iii) chamar o resultado de π
- Evidentemente, o processo especificado precisa terminar com um resultado definido

Função Fatorial

- Outro exemplo de uma definição especificada por um processo é o da **função fatorial**
- Dado um inteiro positivo n , o **fatorial** de n ($n!$) é definido como o produto de todos os inteiros entre n e 1
 - Por exemplo, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$
 - O fatorial de 0 é definido como 1

Função Fatorial: Definição

- Podemos escrever a **definição** dessa função assim:

$$n! = 1 \text{ se } n == 0$$

$$n! = n * (n - 1) * (n - 2) * \dots * 1 \text{ se } n > 0$$

- Os três pontos são abreviatura para todos números entre $n - 3$ e 2 multiplicados

Função Fatorial: Listagem

- Para evitar abreviatura, precisaríamos listar uma fórmula para $n!$ para cada valor de $n!$ separadamente:

$$\begin{aligned}0! &= 1 \\1! &= 1 \\2! &= 2 * 1 \\3! &= 3 * 2 * 1 \\4! &= 4 * 3 * 2 * 1 \\&\dots\end{aligned}$$

- Evidentemente, não esperamos listar uma fórmula para o fatorial de cada inteiro ..
- Para evitar qualquer abreviatura e conjunto infinito de definições, podemos representar um **algoritmo** que aceite um inteiro n e retorne o valor de $n!$

Algoritmo: Fatorial Iterativo

```
1. int fatorial(int n) {  
2.     int x, prod;  
3.  
4.     prod = 1;  
5.     for (x = n; x > 0; x--){  
6.         prod = prod * x;  
7.     }  
8.     return prod;  
9. }
```

- Esse **algoritmo** é chamado **iterativo** porque ele requer a repetição explícita de um processo até que determinada condição seja satisfeita

$$4! = 4 \cdot 3!$$

- Examinaremos a definição de $n!$ que lista uma fórmula separada para cada valor de n
- Podemos observar que $4!$ é igual a $4 \cdot 3 \cdot 2 \cdot 1$, que é igual a $4 \cdot 3!$
- Na verdade, para todo $n > 0$, verificamos que $n!$ é igual a $n \cdot (n - 1)!$

Função Fatorial: Notação Matemática

- Podemos definir:

$$\begin{aligned}0! &= 1 \\1! &= 1 * 0! \\2! &= 2 * 1! \\3! &= 3 * 2! \\4! &= 4 * 3! \\&\dots\end{aligned}$$

- ou, empregando a notação matemática usada anteriormente.

$$\begin{aligned}n! &= 1 \text{ se } n == 0 \\n! &= n * (n - 1)! \text{ se } n > 0\end{aligned}$$

Função Fatorial: Definição Recursiva

- Definimos a função fatorial em termos de si mesma
- Parece uma definição circular .. mas não é
- A notação matemática é tão somente um método conciso de escrever o número infinito de equações necessárias para definir $n!$ para cada n
- Tal definição, que define um objeto em termos de um caso mais simples de si mesmo, é chamada **definição recursiva**

Exemplo 5!

1. $5! = 5 * 4!$
2. $4! = 4 * 3!$
3. $3! = 3 * 2!$
4. $2! = 2 * 1!$
5. $1! = 1 * 0!$
6. $0! = 1$

- Cada caso é reduzido a um **caso mais simples** até chegarmos ao caso de $0!$, que é definido imediatamente como 1
- Podemos, voltar da linha 6 até linha 1, retornando valor calculado em uma linha para avaliar o resultado da linha anterior

Exemplo 5! (Cont.)

$$\begin{array}{l} 6' \quad 0! = 1 \\ 5' \quad 1! = 1 * 0! = 1 * 1 = 1 \\ 4' \quad 2! = 2 * 1! = 2 * 1 = 2 \\ 3' \quad 3! = 3 * 2! = 3 * 2 = 6 \\ 2' \quad 4! = 4 * 3! = 4 * 6 = 24 \\ 1' \quad 5! = 5 * 4! = 5 * 24 = 120 \end{array}$$

Algoritmo: Fatorial Recursivo Incompl.

```
1. int fatorial(int n) {  
2.     int x, y, fact;  
3.  
4.     if (n == 0) {  
5.         fact = 1;  
6.     }  
7.     else {  
8.         x = n - 1;  
9.         // ache o valor de x!. Chame-o de y  
10.        fact = n * y;  
11.    }  
12.    return fact;  
13.}
```

Multiplicação de Números Naturais

- Outro exemplo de **definição recursiva** é a multiplicação de números naturais
- O produto $a \cdot b$, em que a e b são inteiros positivos, pode ser definido como a somado a si mesmo b vezes. Essa é uma **definição iterativa**
- Uma **definição recursiva** equivalente é

$$a * b = a \text{ se } b == 1$$

$$a * b = a * (b - 1) + a \text{ se } b > 1$$

$$mult(a, b) = \begin{cases} a & \text{se } b = 1 \\ mult(a, b - 1) + a & \text{se } b > 1 \end{cases}$$

Multiplicação de Números Naturais (Cont.)

- Para avaliar $6 \cdot 3$ por meio dessa definição, avaliamos primeiro $6 \cdot 2$ e depois somamos 6
- Assim:
$$6 \cdot 3 = 6 \cdot 2 + 6 = 6 \cdot 1 + 6 + 6 = 6 + 6 + 6 + 6 = 18$$

Padrão em Definições Recursivas

- Observe um padrão:
 - Um **caso simples** é estabelecido explicitamente
 - No caso do fatorial, $0!$ foi definido como 1 ; no caso da multiplicação, $a \cdot 1 = a$
 - Os outros casos são definidos aplicando uma operação sobre o resultado da **avaliação de um caso mais simples**
 - $n!$ é definido em termos de $(n - 1)!$ e $a \cdot b$ em termos de $a \cdot (b - 1)$

Simplificações → Caso Trivial

- Eventualmente, simplificações sucessivas de determinado caso devem levar ao **caso trivial**
 - **Caso trivial** é definido de maneira explícita
 - No caso da função fatorial, subtrair 1 de n , várias vezes, resulta eventualmente em 0
 - No caso da multiplicação, subtrair 1 de b , várias vezes, resulta eventualmente 1
- Seria inválido se definíssemos:
$$n! = (n + 1)! / (n + 1)$$

ou
$$a * b = a * (b + 1) - a$$

(Tente determinar $5!$ ou $6 \cdot 3$ usando as definições anteriores)

Sequência de Fibonacci

- A **sequência de Fibonacci** é a sequência de inteiros:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...
- Cada elemento nessa sequência é a soma dos dois anteriores
 - Por exemplo, $0 + 1 = 1$, $1 + 1 = 2$, $1 + 2 = 3$, $2 + 3 = 5$, ...
- Dado $fib(0) = 0$ e $fib(1) = 1$, podemos definir com definição recursiva:

$$fib(n) = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ fib(n-2) + fib(n-1) & \text{se } n \geq 2 \end{cases}$$

fib(5)

- Para calcular *fib*(5), por exemplo, podemos aplicar a definição recursivamente para obter:

$$fib(6) = fib(4) + fib(5) = fib(2) + fib(3) + fib(5) = \dots$$

(Quadro)

Redundância Computacional

- Definição recursiva dos números de Fibonacci refere-se a si mesma duas vezes
 - Muita redundância computacional
- Método iterativo *fib*(*n*) é muito mais eficiente
 - Compare o número de adições (não incluindo os incrementos da variável índice *i*) efetuadas ao calcular *fib*(6) por meio desse algoritmo e usando a definição recursiva

Busca & Busca Linear

- Recursividade vai além de definir funções matemáticas
- Por exemplo, queremos pesquisar um nome em um catálogo de telefones, uma palavra num dicionário ou determinado empregado num arquivo de pessoal
 - O processo usado para encontrar tal elemento é chamado **busca**
- O método de busca mais simples é o da **busca linear**
 - No qual cada item do vetor é examinado por vez e comparado ao item que se está procurando

Lista Não Ordenada ou Ordenada

- Se a **lista está desorganizada**, **busca linear** pode ser única maneira de localizar algo dentro dela
- Entretanto, um método desse tipo jamais seria usado para localizar um nome num catálogo de telefones
- Em vez disso, o catálogo seria aberto numa página qualquer e os nomes pertencentes a essa página seriam examinados
- Como os nomes encontram-se em **ordem alfabética**, esse exame determinaria se a busca deve continuar na primeira ou segunda metade do catálogo

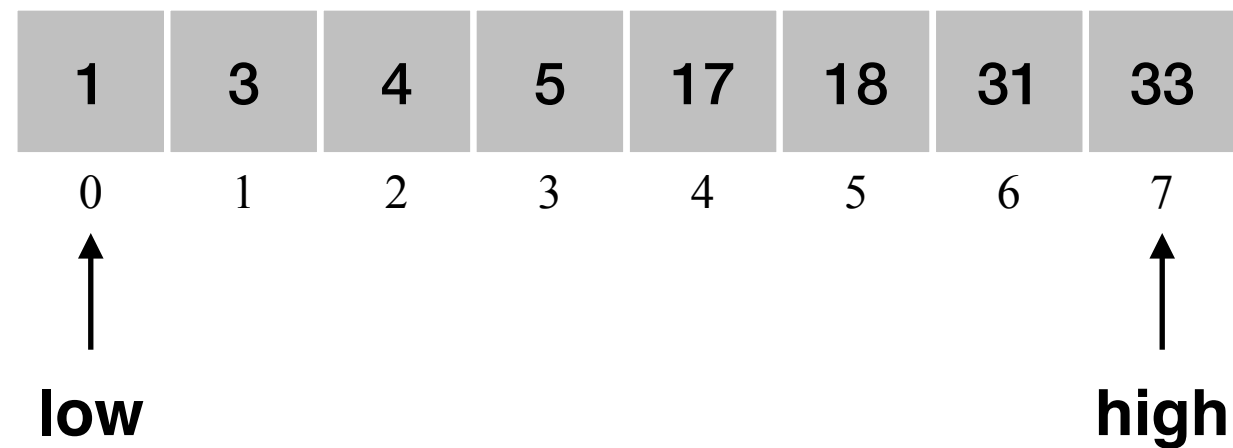
Busca Binária

- Se vetor possuir um elemento, problema simples
- Caso contrário, compare item procurado ao item posicionado no **meio** do vetor
 - Se iguais, busca terminou com sucesso
 - Se **elemento do meio** for **maior** que o **elemento procurado**, o processo de busca será repetido na primeira metade do vetor
 - Caso contrário, o processo será repetido na segunda metade
- Esse método de busca é conhecido como **busca binária**

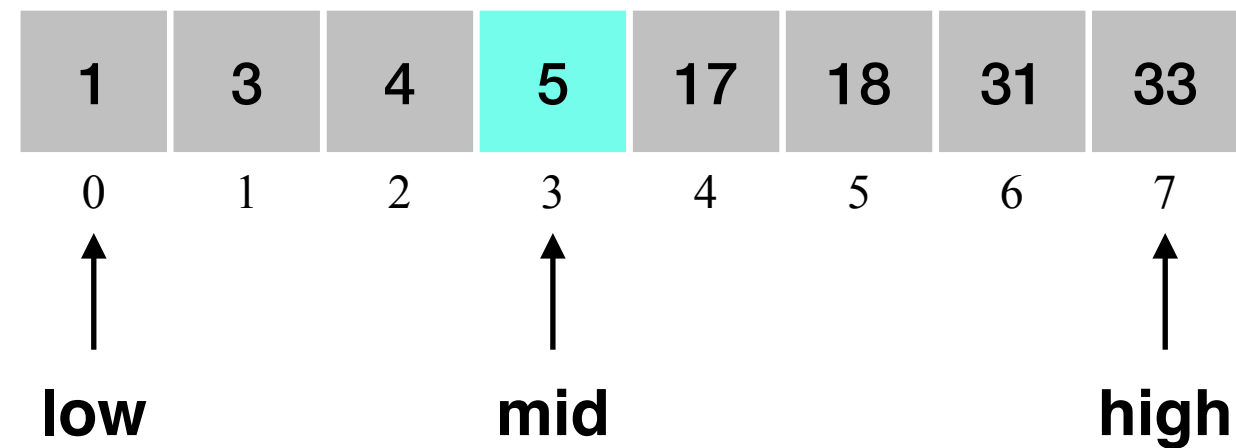
Algoritmo: Busca Binária

```
1. if (low > high)
2.     return -1;
3. mid = (low + high) / 2;
4. if (x == a[mid])
5.     return mid;
6. else if (x < a[mid])
7.     ; // procura x em a[low] ateh a[mid - 1];
8. else
9.     ; // procura x em a[mid + 1] ateh a[high];
```

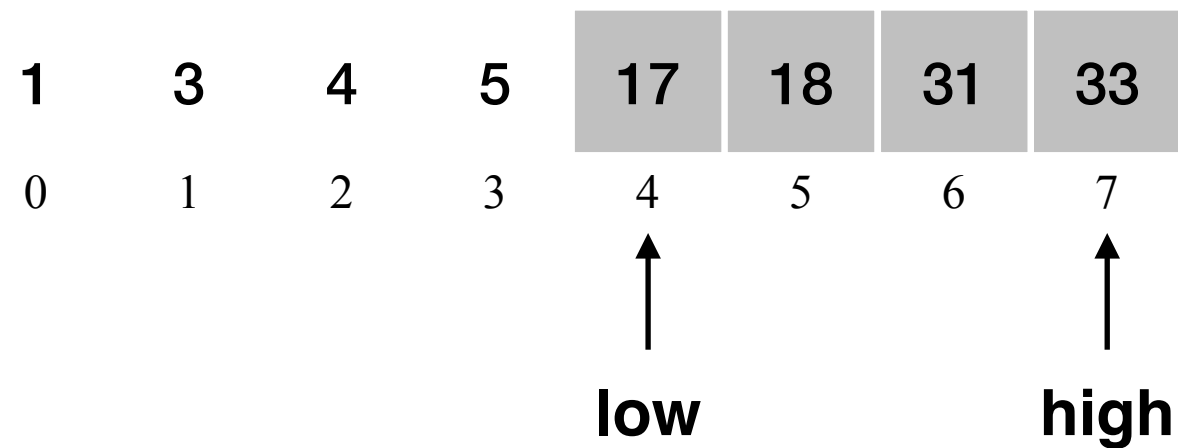

Busca Binária: Procurar Elemento 17 (1/5)



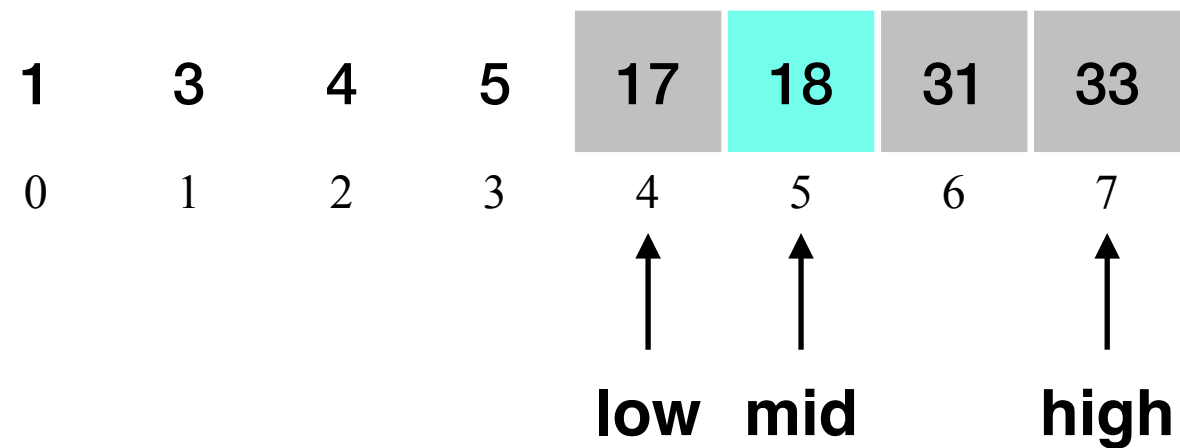
Busca Binária: Procurar Elemento 17 (2/5)



Busca Binária: Procurar Elemento 17 (3/5)

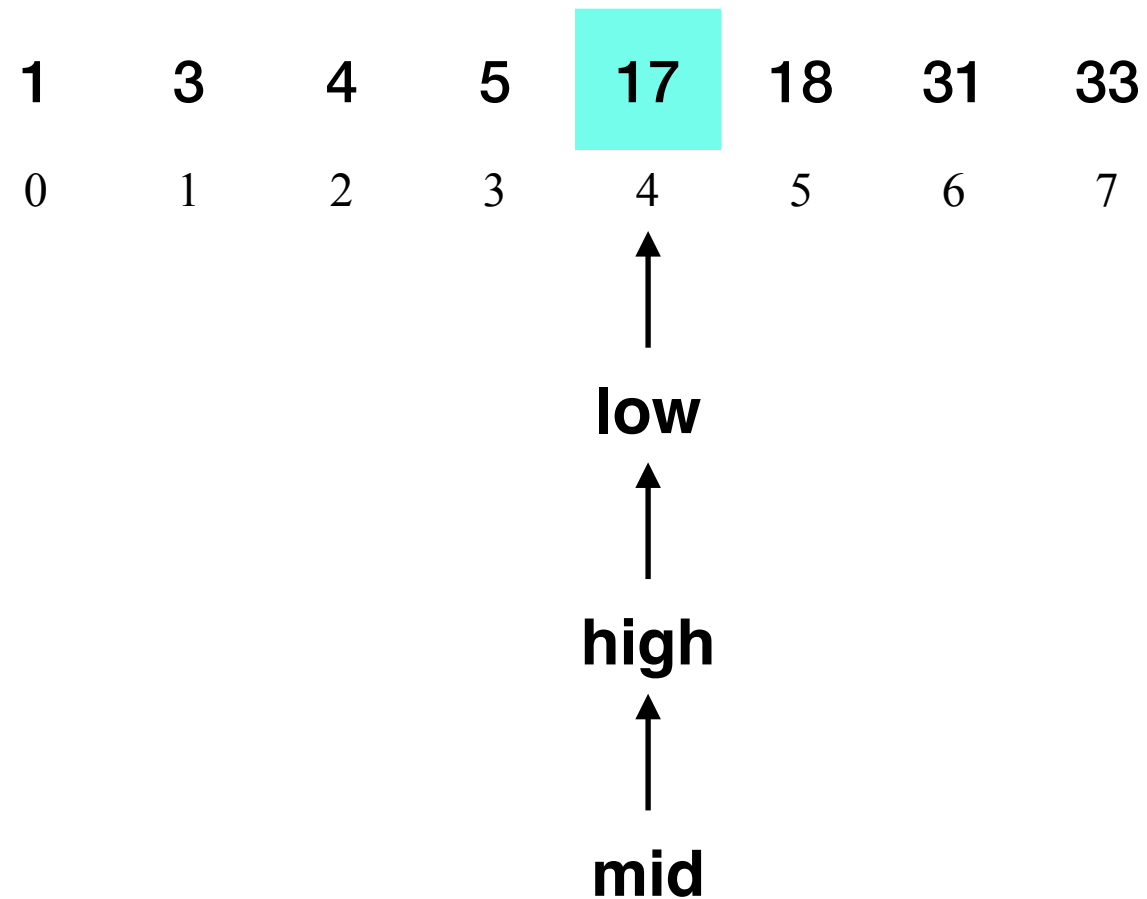


Busca Binária: Procurar Elemento 17 (4/5)

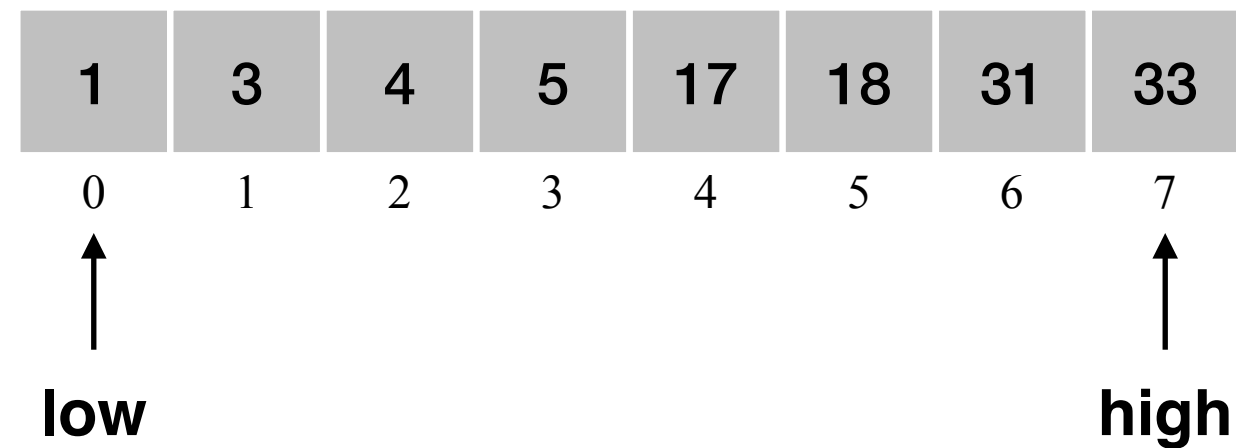


Busca Binária: Procurar Elemento 17 (5/5)

Como $a[4] == 17$, retorne $mid = 4$ como resposta.



Busca Binária: Procurar Elemento 2



Propriedades Algoritmos Recursivos

- Não gere **sequência infinita de chamadas** a si mesmo
- Para, no mínimo, um argumento ou grupo de argumentos, a função recursiva f deve ser definida de modo a não envolver f
 - Deve existir uma **“saída”** da sequência de chamadas recursiva
 - Fatorial: $0! = 1$
 - Multiplicação: $a \cdot 1 = a$
 - Seq de Fibonacci: $fib(0) = 0; fib(1) = 1$
 - Busca binária: `if (low > high) return -1; if (x == a[mid]) return (mid);`

Recursividade em C

- A linguagem C permite que um programador escreva sub-rotinas e funções que chamem a si mesmas
- Tais rotinas são denominadas **recursivas**

Algoritmo: Fatorial Recursivo

```
1. int fatorial(int n) {  
2.     int x, y, fact;  
3.  
4.     if (n == 0) {  
5.         fact = 1;  
6.     }  
7.     else {  
8.         x = n - 1;  
9.         // ache o valor de x!. Chame-o de y  
10.        y = fatorial(x);  
11.        fact = n * y;  
12.    }  
13.    return fact;  
14.}
```


Fatorial(4)

fatorial(4)

```
int fatorial(int n=0) {  
    int fact, x, y;  
  
    if (n == 0) {  
        fact = 1;  
    }  
    else {  
        x = n - 1;  
        // ache o valor de x!. Chame-o de y  
        y = fatorial(x);  
        fact = n * y;  
    }  
    return fact;  
}
```

Fatorial(4): Retorno

fatorial(4)

The diagram illustrates the recursive calls for the function fatorial(4). It consists of four overlapping rectangular frames, each representing a function call. The frames are stacked such that the one for fatorial(4) is at the back, and the one for fatorial(0) is at the front. Each frame contains the function definition. The return value for each call is shown in a yellow box at the bottom of the frame. The return values are 1 for fatorial(0), 1 for fatorial(1), 2 for fatorial(2), and 6 for fatorial(3). The final return value for fatorial(4) is 24, which is shown in a yellow box at the bottom of the frontmost frame.

```
int fatorial(int n=0) {  
    int fact, x, y;  
  
    if (n == 0) {  
        fact = 1;  
    }  
    else {  
        x = n - 1;  
        // ache o valor de x!. Chame-o de y  
        y = fatorial(x);  
        fact = n * y;  
    }  
    return fact=1;  
}
```

Fatorial(4): Retorno

fatorial(4)

```
int fatorial(int n=1) {  
    int fact, x, y;  
  
    if (n == 0) {  
        fact = 1;  
    }  
    else {  
        x = n - 1;  
        // ache o valor de x!. Chame-o de y  
        y = fatorial(x)=1;  
        fact = n * y;  
    }  
    return fact=1;  
}
```

Fatorial(4): Retorno

fatorial(4)

```
int fatorial(int n=2) {  
    int fact, x, y;  
  
    if (n == 0) {  
        fact = 1;  
    }  
    else {  
        x = n - 1;  
        // ache o valor de x!. Chame-o de y  
        y = fatorial(x)=1;  
        fact = n * y;  
    }  
    return fact=2;  
}
```

Fatorial(4): Retorno

fatorial(4)

```
int fatorial(int n=3) {  
    int fact, x, y;  
  
    if (n == 0) {  
        fact = 1;  
    }  
    else {  
        x = n - 1;  
        // ache o valor de x!. Chame-o de y  
        y = fatorial(x)=2;  
        fact = n * y;  
    }  
    return fact=6;  
}
```

Fatorial(4): Retorno

fatorial(4)

```
int fatorial(int n=4) {  
    int fact, x, y;  
  
    if (n == 0) {  
        fact = 1;  
    }  
    else {  
        x = n - 1;  
        // ache o valor de x!. Chame-o de y  
        y = fatorial(x)=6;  
        fact = n * y;  
    }  
    return fact=24;  
}
```

Pilha Durante a Execução Fatorial

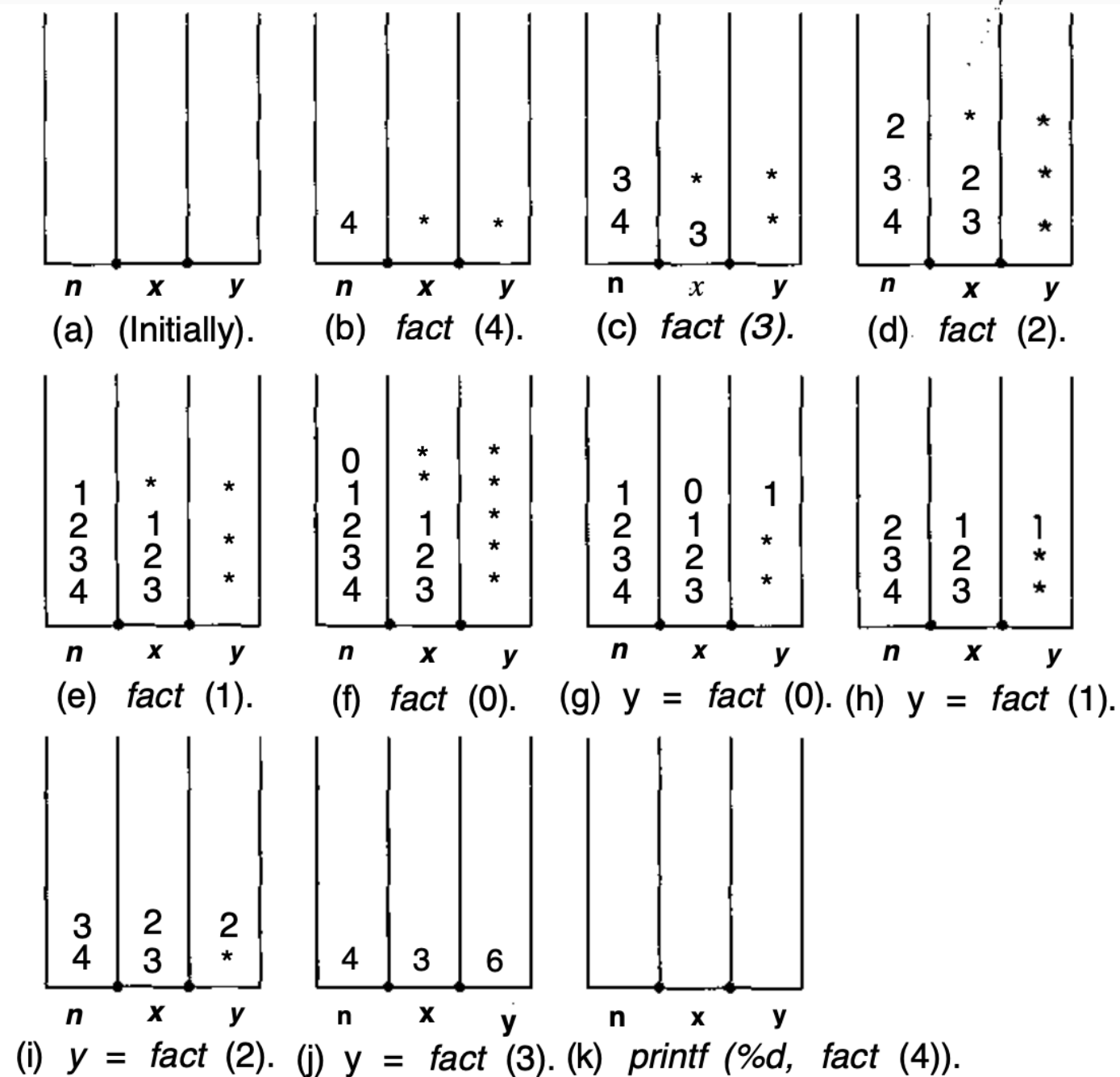


Figura 32.1 A pilha, em vários instantes, durante a execução. (Um asterisco indica um valor não-inicializado.)

Algoritmo: Multiplicação Recursivo

```
1. int mult(int a, int b) {  
2.     return (b == 1) ? a : mult(a, b - 1) + a;  
3. }
```

ou

```
1. int mult(int a, int b) {  
2.     int c, d, sum;  
3.  
4.     if (b == 1) {  
5.         return a;  
6.     }  
7.     c = b - 1;  
8.     d = mult(a, c);  
9.     sum = d + a;  
10.    return sum;  
11. }
```


Algoritmo: Fibonacci Recursivo

```
1.int fib(int n) {  
2.    int x, y;  
3.  
4.    if (n <= 1)  
5.        return n;  
6.    x = fib(n - 1);  
7.    y = fib(n - 2);  
8.    return x + y;  
9.}
```

Pilha Durante a Execução Fibonacci

Algoritmo: Busca Binária Recursivo

```
1. int busca_binaria(int a[], int x, int low, int high) {
2.     int mid;
3.
4.     // calcule a posicao do meio
5.     mid = (low + high) / 2;
6.
7.     // se inicio > fim, entao busca acabou sem encontrar o x
8.     if (low > high)
9.         return -1;
10.    // se x esta na posicao do meio
11.    else if (x == a[mid])
12.        return mid;
13.    // se x eh menor que elemento do meio
14.    else if (x < a[mid])
15.        return busca_binaria(a, x, low, mid - 1);
16.    // se x eh maior que elemento do meio
17.    else
18.        return busca_binaria(a, x, mid + 1, high);
19.}
```

Referências

- TENENBAUM, Aaron M; ANGENTEIN, Moshe; LANGSAM, Yedidiah. Estruturas de dados usando C. Sao Paulo, SP: Pearson, 1995. 884p.
- FEOFILOFF, Paulo. Algoritmos em linguagem C. Rio de Janeiro: Elsevier, 2009. 208 p.